# Cooperative Service Caching in Vehicular Edge Computing Networks Based on Transportation Correlation Analysis

Chen Ling, Weizhe Zhang, *Senior Member, IEEE*, Qingyang Fan, Zebang Feng, Jiayin Wang, Rahul Yadav, and Desheng Wang

***Abstract*—In vehicular edge computing, the vehicular services are cached and replaced among roadside units (RSUs) to minimize delay in delay-sensitive services. However, without prior knowledge of these service preferences, the cached services suffer a low-hit ratio and a high-network delay. Before the appearance of requests from vehicles, most existing vehicular service caching methods design the service-sharing mechanisms among RSUs to optimize the hit ratio, but they ignore the influence of vehicle mobility on service-sharing. Moreover, in vehicular services replacement, the differences between actual and historical vehicle trajectories negatively impact the service-sharing mechanisms constructed before, which existing methods ignored. Thus, this article addresses the vehicular service caching problem based on the surrounding function-features of vehicles and transportation correlations. First, we use the surrounding function-features of vehicles to estimate the service preference and ensure the hit ratio of cached services. Second, we formulate the vehicular service caching problem as a constrained optimization problem and design a cooperation mechanism between RSUs, which is based on transportation correlations of vehicle trajectories. Third, we propose two vehicular service caching methods based on Gibbs sampling to optimize the network delay of vehicular services and deal with the negative influence of vehicle mobility. Simulation in real data sets from Shenzhen, China shows that our methods have a better network delay and hit ratio than existing baselines by 15.26% and 5.6%, respectively.**

***Index Terms*—Edge computing, heuristic algorithm, Internet of Vehicle (IoV), network delay optimization, service caching.**

Manuscript received 5 December 2023; revised 27 January 2024 and 8 March 2024; accepted 26 March 2024. Date of publication 2 April 2024; date of current version 7 June 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2022ZD0115303. *(Corresponding author: Weizhe Zhang.)*

Chen Ling is with the School of Computer Science and Technology, Harbin Institute of Technology, Harbin 150001, China (e-mail: 18B903060@stu.hit.edu.cn).

Weizhe Zhang is with the School of Computer Science and Technology, Harbin Institute of Technology, Harbin 150001, China, and also with the Department of New Networks, Peng Cheng Laboratory, Shenzhen 518066, Guangdong, China (e-mail: wzzhang@hit.edu.cn).

Qingyang Fan and Jiayin Wang are with the School of Cyberspace Science, Harbin Institute of Technology, Harbin 150001, China (e-mail: 21B903073@stu.hit.edu.cn; 44957602@qq.com).

Zebang Feng is with the Navigation Department, NavInfo Company Ltd., Beijing 100094, China (e-mail: fengzebang8336@navinfo.com).

Rahul Yadav is with the College of Computer Science and Technology, Harbin Engineering University, Harbin 150009, China (e-mail: rahul@hrbeu.edu.cn).

Desheng Wang is with the School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), Shenzhen 518055, Guangdong, China (e-mail: wangdesheng@hit.edu.cn).

Digital Object Identifier 10.1109/JIOT.2024.3382723

## I. Introduction

THE VIGOROUS development of the Internet of Vehicle (IoV) enables diverse vehicular services in vehicular networks, such as intelligent vehicle control, environment-aware, and in-vehicle entertainment [1]. Most vehicular services are computation-intensive and delay-sensitive, requiring massive computation and network resources. Unfortunately, due to the resource-constrained hardware condition of vehicles, the vehicles cannot provide sufficient resources to meet the requirements of vehicular services. Therefore, the researchers introduced vehicular-mobile edge computing (MEC) to overcome resources problem in vehicles via roadside units (RSUs) [2]. The RSU deployed close to vehicles in vehicular-MEC, providing computation and network resources to vehicles and alleviating vehicle overhead [3], [4]. In vehicular-MEC architectures [2], sufficient resources of RSUs help to minimize the overhead of vehicle's computation-intensive and delay-sensitive services [5], [6].

In urban areas, the preferences of vehicular services running on RSUs are highly influenced by the surrounding function-features of vehicles [7]. After receiving the requests from vehicles, RSUs fetched the related programs and data requested by vehicular services from the cloud data center and forwarded the service results to vehicles. As a result, it increases the service delay. Therefore, caching the vehicular services on RSUs based on service preferences avoids fetching delays occur by requested programs and data, as shown in Fig. 1. If the vehicles' requests hit the cached services, RSUs will generate service results locally and eliminate the fetching delay. Traditionally, vehicular service caching method refers to caching all vehicular services requested by vehicles on each RSU. However, due to limited resources of RSUs, this method can lead to occur resources overloaded situation. Therefore, future vehicular-MEC architectures require an efficient caching method to allocate resources to vehicular services.

### A. Motivations and Problems

Some researchers have formulated vehicular service caching methods to optimize network delay [5], energy

2327-4662 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

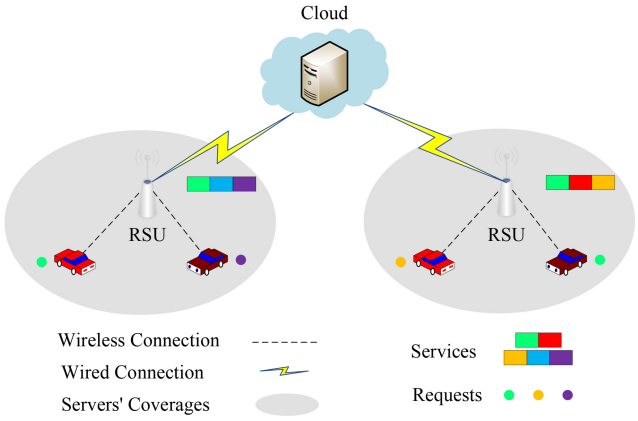



Fig. 1. Scene of vehicular service caching.

consumption [8], [9], and quality of experience [10]. In detail, most vehicular service caching methods include offline and online stages. The offline stage is before the appearance of requests from vehicles, where vehicular service caching methods cache the vehicular services on RSUs based on history service preferences, services properties, and RSUs' resources. Specifically, Zeng et al. [5], Shen et al. [8], Song et al. [10], Mondal et al. [11], and Malik et al. [12] cached the most popular vehicular services to maximize the utilization of cached services and propose offline vehicular service caching methods. Differently, some other works [13], [14], [15], [16], [17] cache vehicular services in RSUs near to vehicles to further minimize the network delay of vehicular services. Then, in urban areas, the vehicles' requests for service are time-varying. Specifically, some of the services already cached on RSUs might be infrequently requested, which results in decreasing the utilization of cached services on RSUs. Therefore, to ensure the utilization of service caches, the researchers in [18], [19], [20], and [21] replace the infrequently requested services with frequently requested services during the online stage. Without the replacement mechanism during the online stage, the cached services may suffer low-utilization and high-network delay. However, existing vehicular service caching methods show a high-network delay and a low-hit ratio in real-world circumstances.

*The Drawbacks of the Existing Vehicular Service Caching Methods:* First, most caching methods require real-time data sets of service preferences to maximize the hit ratio of cached services [12], [22]. However, most existing caching methods used synthetic data sets that are incompatible with real-world problems. Second, during the offline stage, some researchers introduced the service-sharing mechanism to develop cooperative vehicular service caching methods. These methods could share the cached services among RSUs to increase the hit ratio of the service cache. However, one RSU can only share vehicular services with other RSUs within its communication coverages in existing methods, resulting in a low-hit ratio and high-network delay [5]. The vehicles with long trajectories may pass through several RSUs' communication coverages, whose requests cannot utilize the service-sharing mechanism of existing methods. Third, during the online stage, vehicle mobility changes the service preferences, decreasing the hit ratio of cached services. Moreover, the actual trajectory of the vehicle may be different from the historical trajectory. The service-sharing among RSUs in history trajectories may be incompatible with the actual trajectories. However, replacement mechanisms in existing methods only consider the changed service preferences but ignore the influence of vehicle mobility on service-sharing among RSUs.

These drawbacks motivate us to propose a new vehicular service caching method. However, there are three main challenges. First, the vehicular service caching method requires real data sets of service preferences. Therefore, the first challenge of our work is *CH1:* How to estimate the service preferences close to the real-world scenario of IoV environments using real data sets? Second, during the offline stage, vehicular service caching methods construct the service-sharing mechanism among RSUs to optimize the hit ratio and network delay of cached services. Moreover, the service-sharing mechanism should consider both RSUs' communication coverages and history vehicle trajectories to increase the possibility of service-sharing. So that the second challenge of our work is *CH2:* How to design a cooperation mechanism during the offline stage that can maximize the advantages of the service-sharing among RSUs? Third, the differences between actual and historical vehicle trajectories play a significant role in influencing service-sharing among RSUs. Moreover, during the online stage, the replacement mechanism in the vehicular service caching method should consider both the changed service preferences and the influence of vehicle mobility to optimize the hit ratio of cached services. Therefore, the third challenge is *CH3:* How to design a vehicular service cache replacement mechanism during the online stage which can minimize the influence of vehicle mobility on the cooperation among RSUs?

### B. Contributions

This article proposes two vehicular service caching methods to optimize cached services' hit ratio and network delay. First, to take vehicle mobility into service-sharing mechanism's account, we propose an analyzation method of transportation correlation to measure the effect of service-sharing. Based on this analyzation method, the proposed methods could optimize the cached services' hit ratio. Second, we further design a vehicular service cache replacement mechanism in proposed methods. Detailly, considering the influence of vehicle mobility on service-sharing, our methods replace the infrequently requested services with frequently requested services. The key contributions of our work are summarized as follows.

1) Targeting *CH1*, we propose a service preference estimation method based on the surrounding function-features of vehicles to estimate the service preferences. Moreover, these service preferences correlate with real-time vehicle positions. So, the proposed method estimates the service preferences close to the real-world scenario of IoV environments.
2) Targeting *CH2*, we propose a transportation-correlation-based vehicular service caching method during the

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

offline stage named "transportation correlations-based caching strategy (TCCS)." In TCCS, we analyze the transportation correlation based on vehicle trajectories and cache the vehicular services on RSUs as per service preferences. Specifically, the transportation correlation between each road is analyzed based on the vehicular history trajectories, representing the probability of service-sharing among RSUs. Then, an expected profit is proposed to evaluate the performance of cached services based on transportation correlations. Finally, we propose TCCS to take full advantage of the service-sharing mechanism and optimize the network delay and hit ratio of cached services.

3) Targeting *CH3*, we design a vehicular service caching replacement mechanism during the online stage named "dynamic cache replacement strategy (DCRS)." DCRS is based on the changes in service preference and transportation correlation, calculated based on vehicle-service requests received by RSUs. In detail, DCRS finds the infrequently requested services based on the request number of vehicular services. Then, DCRS calculates the expected profits of uncached vehicular services and selects the services with maximum expected profits. Finally, DCRS replaces the infrequently requested services with uncached services based on changed transportation correlation to increase expected profits and the hit ratio of cached services.
4) The simulation experiments conducted on a real-world data set show that our proposed methods, TCCS and DCRS, optimize the hit ratio and network delay by 15.26% and 5.6% relatively compared with existing baselines.

This article is organized as follows. Section II describes related work on vehicular service caching methods. Section III describes our problem model of vehicular service caching. In Section IV, we describe the procedure of TCCS. Then, Sections V and VI introduce the problem model of dynamic service cache replacement and the procedure of DCRS. Afterward, in Section VII, we evaluate the performance of TCCS and DCRS in terms of the hit ratio and network delay. Finally, Section VIII summarizes our work and draws some conclusions.

## II. Related Work

In this section, we summarize the research on vehicular service caching method. The research includes two categories: 1) mobility awareness-based and 2) resource management-based [23]. The research based on mobility awareness caches the services based on the current and future position of vehicles. On the other hand, the research based on resource management focuses more on the resource requirement of services and the resource allocation of RSUs. Detailed research on these two categories is introduced below.

### A. Research Based on Mobility Awareness

The mobility awareness-based service caching methods first predict the vehicles' trajectory based on their history trajectories. Then, considering the vehicles' future positions and service preference, the service caching methods cache the services on the RSUs close to vehicles to optimize the hit ratio and network delay. Li et al. [13] developed a cooperative service caching method based on Gibbs sampling and deep reinforcement learning to optimize the quality of service. However, the proposed method did not be verified in real-world data sets. Li and Wu [14] jointly formulated the problem model of computation offloading and data caching based on vehicle mobility, edge servers' resources, task requirements, and user preferences. Then, they proposed two heuristic algorithms to solve the problem model and optimize applications' completion delay and Quality of Service. However, the service cache replacement is ignored in their algorithms. Zhang et al. [15] utilized the Terahertz frequency link in vehicular content caching and resource allocation in vehicular networks to optimize energy efficiency. Nevertheless, they did not verify the influence of different content caching methods in their method. Zhang et al. [18] proposed a hierarchical proactive caching method based on service preferences and mobility to optimize the hit ratio and average hop number of cached videos. Nevertheless, they ignored the cooperation among RSUs and base stations, decreasing the hit ratio of cached videos. Mahmood et al. [16] proposed a mobility-based content cache scheme to prefetch the content from edge nodes and maximize the throughput and hit ratio of cached contents in vehicular networks. However, the unstable vehicular speed may influence the mobility prediction accuracy. Du et al. [24] proposed a multiagent deep deterministic policy gradient (MADDPG)-based algorithm to enable low-complexity and real-time task offloading of vehicles. They offload the tasks of vehicles to the unmanned aerial vehicles (UAVs) to optimize the offloading delay and costs.

Wu et al. [25] proposed a service provision method based on deep reinforcement learning and mobility prediction to optimize the delay of the service on edge nodes. Nevertheless, the service preference is ignored in the proposed method. Yu et al. [20] proposed a mobility-aware edge caching scheme based on federated learning to cooperatively cache and replace the content among vehicles to decrease the network delay and system cost. However, connections between vehicles are too vulnerable to support the proposed scheme. Huang et al. [26] proposed a task offloading and resource allocation method based on vehicle speed aware to optimize energy consumption and guarantee the delay constraint. However, the vehicle speed may vary in the offloading processing, influencing the network delay and energy consumption. Kyung and Song [17] introduced vehicular caching into the task offloading problem. They designed a content service offloading system to offload and migrate the content services, decreasing the overhead of edge servers. However, the proposed algorithms did not consider the service interruption caused by vehicle mobility. AlNagar et al. [27] considered the cooperation among RSUs in content delivery to accelerate the transfer of information to users. They proposed a content caching algorithm to cache the contents on each RSU with low-network delay. However, the cooperation of RSUs can be influenced by vehicle mobility, which the proposed algorithm ignores. Wu et al. [21] proposed

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

a cooperative service caching and task scheduling method to allocate resources of edge servers and vehicles to vehicular services. However, they ignore the replacement of cached services. Yu et al. [19] proposed a mobility-aware proactive edge caching method based on the mobility prediction of vehicles. Nevertheless, the preferences of different vehicular services are ignored in this method.

### B. Research Based on Resource Management

The research based on resource management mainly allocates the resources RSUs possess to the services requested by vehicles. Considering the service preference of vehicles, the RSUs cache the most popular services to ensure the hit ratio of services. Finally, the RSUs could share the cached services with others to increase the hit ratio. Zeng et al. [5] proposed a vehicular service caching framework based on Software Define Network, where edge servers cooperatively cache services based on the density of requesting vehicles. However, the cooperation among edge servers of proposed framework ignores the influence of vehicle mobility. Shen et al. [8] developed a semi-distributed algorithm based on dynamic programming to solve the dependency-aware task offloading and service caching problem jointly. However, they ignored the influence of vehicular mobility on offloading completion ratio. Song et al. [10] proposed a caching method based on deep reinforcement learning for vehicular-MEC environments to optimize the performance of short video systems. Nevertheless, the cooperation among edge servers is ignored in their caching method, decreasing the hit ratio of cached videos. Mondal et al. [11] proposed a game-theoretic service caching scheme, CALM, for multisensor-cloud to optimize the delay of vehicular sensor-cloud service in internal and external stages. However, CALM ignores the sharing of cached contents among RSUs, decreasing the hit ratio of cached contents. Zhang et al. [28] proposed a light-weight cryptography-based access control scheme to preserve the privacy of cached contents. Nevertheless, they did not verify in real-world data sets and ignored the influence of vehicular mobility. Gu et al. [29] designed a model-assisted DRL framework for VNF orchestration to optimize the network performance of datacenters. However, they ignored the mobility problem of users.

Elgendy et al. [30] jointly formulated a reinforcement model for computation offloading and task caching. They proposed two $Q$-learning and deep-$Q$-network-based algorithms to derive the optimal offloading and caching decisions and minimize the system overhead. However, the mobility problem is ignored in deriving offloading and caching decisions. Choi et al. [31] proposed a video caching method based on deep deterministic policy gradient to cache the video file on vehicles and optimize the energy consumption and video quality. However, they did not consider the video preference in different areas. Liu et al. [32] proposed an online approach based on deep $Q$ networks to jointly cache services, allocate resources, and minimize the total delay of computation tasks. Nevertheless, they did not verify the proposed approach in real-world environments. Gu et al. [33] formulated the microservice placement and request scheduling as an optimization problem and proposed an iterative greedy algorithm based on layer sharing to minimize edge servers' overhead and network delay. However, the proposed algorithms focus more on container-based microservices, which may be incompatible with other microservices like virtual machines. Ren et al. [34] proposed an online twin-timescale framework to schedule the requests and cache the services based on deep reinforcement learning. However, they ignored service interruption caused by user mobility. Gu et al. [35] proposed a two-tier data center architecture that leverages the excessive storage resources in parking lots to mitigate the pressure on the conventional data center. Moreover, they also proposed three VDC management policies and derived their total communication cost in closed form. Du et al. [36] proposed a resource pricing and allocation method for blockchain systems based on asynchronous advantage actor-critic (A3C) deep reinforcement learning. The proposed method allocate and price the resources of edge servers to maximize the profits of the miners in blockchain systems.

Malik et al. [12] studied the content-centric mobile network and proposed a preference learning-based service cache framework, which compares the preferences between contents and caches the contents to maximize the hit ratio of cached content. Nevertheless, they did not verify the proposed framework in real-world trajectory data sets. Zong et al. [37] proposed a dynamic and heterogeneous content caching method based on ensemble learning to optimize the network delay of edge computing environments. However, the proposed framework needs to collect multiple user information, creating high-network overheads. Dai et al. [38] studied the dynamic service caching for augmented Intelligence of Things devices in enterprise management systems and proposed distributed task offloading and service caching algorithms to optimize the network delay and energy consumption. Nevertheless, ignoring the cooperation among edge servers in the proposed algorithms caused a low-hit ratio. Nguyen et al. [22] proposed a proactive caching scheme based on user preference using long short-term memory networks to maximize the hit ratio of cached content. However, the proposed scheme requires long-term request data sets to predict user preference, causing high-network overheads to collect these data sets. Degan et al. [39] introduced the content source choice mechanism into the content caching problem in IoV and proposed a content distribution method based on the immune cloning method. However, the proposed method heavily depends on the accuracy of preference prediction, which is a challenge in the vehicular caching problem. Gu et al. [40] proposed a low-complexity online-distributed algorithm using the Lyapunov optimization framework to process the network traffic burst and allocate network resources. Moreover, they evaluated the proposed algorithm in both simulation and testbed-based experiments.

In conclusion, these related works have some critical limitations.

1) Existing vehicular service caching methods heavily depend on service preferences. However, these methods are verified by randomly generated

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOTATIONS USED IN THIS ARTICLE

| Notation | Meaning |
|---|---|
| $V$ | Vehicle set |
| $R$ | RSU set |
| $r_i$ | An RSU |
| $f_i$ | CPU computing capability of $r_i$ |
| $m_i$ | Memory capacity of $r_i$ |
| $h_i$ | Hard-disk drive capacity of $r_i$ |
| $v_j$ | A vehicle |
| $route_j$ | Trajectory of $v_j$ |
| $l_m$ | A road |
| $A_i$ | The set of roads within $r_i$'s coverage |
| $A'_m$ | The set of roads correlate with $l_m$ |
| $S$ | Service library |
| $t_j$ | A service request |
| $s_k$ | A service |
| $cpu_k$ | Computing resource requirement of $s_k$ |
| $mem_k$ | Memory resource requirement of $s_k$ |
| $hard_k$ | Hard-disk drive resource requirement of $s_k$ |
| $P^I_m$ | Number of vehicles pass through the road $l_m$ |
| $P^F_{mk}$ | Service preference of category $k$ of road $l_m$ |
| $P^{con}_{ab}$ | The transportation correlation of roads $l_a$ and $l_b$ |
| $P^{con}_{ab\tau}$ | The transportation correlation of roads $l_a$ and $l_b$ in time frame $\tau$ |
| $P^{con}_{\tau}$ | The transportation correlation set of all roads in time frame $\tau$ |
| $P^{hit}_{ik\tau}$ | The hit ratio of service $s_k$ in time frame $\tau$ |
| $loc_{mk}$ | A POI record of category $k$ around road $l_m$ |
| $C$ | Service category Set |
| $T_{im}$ | The transmission rate between $l_m$ and $r_i$ |
| $T_D$ | The maximum transmission rate |
| $T_C$ | The transmission rate between RSUs and the cloud |
| $P_t$ | The transmission power of RSUs and vehicles |
| $P_n$ | The noise power of RSUs and vehicles |
| $D(l_m, r_i)$ | The distance between road $l_m$ to RSU $r_i$ |
| $delay^H_{imk}$ | The network delay when requested services are cached |
| $delay^M_{imk}$ | The network delay when requested services are not cached |
| $E^{local}_{imk}$ | The local expected profit of service $s_k$ at RSU $r_i$ |
| $E^{shared}_{imk}$ | The shared expected profit of service $s_k$ at RSU $r_i$ |
| $E_{ik}$ | The total expected profit of service $s_k$ at RSU $r_i$ |
| $E^{\tau}_{ik}$ | The service cache replacement profit of service $s_k$ at RSU $r_i$ in time frame $\tau$ |
| $e^{\tau}_{ik}$ | The service status of service $s_k$ at RSU $r_i$ in time frame $\tau$ |
| $f^{\tau}_k$ | Released CPU computing capability of $r_k$ in time frame $\tau$ |
| $m^{\tau}_k$ | Released memory capacity of $r_k$ in time frame $\tau$ |
| $h^{\tau}_k$ | Released hard-disk drive capacity of $r_k$ in time frame $\tau$ |
| $\Gamma$ | Time frame set |
| $\tau$ | A Time frame |
| $\alpha$ | The weight coefficient |
| $\varepsilon$ | The smooth parameter |
| $strategy$ | The service caching strategy of all RSUs |
| $strategy^{\tau}$ | The service caching strategy of all RSUs in time frame $\tau$ |

service preferences, which are far from the real world.

2) The vehicle mobility is not deeply considered in existing vehicular service caching methods. The vehicle mobility influences the cooperation among RSUs, limiting the optimization of the service-sharing mechanism.
3) Most vehicular service caching methods ignore the service cache replacement mechanism. Existing vehicular-MEC architectures need to replace infrequently requested services with frequently requested services to increase hit ratio.

## III. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we briefly describe the system model of our vehicular service cache system and give a problem formulation of vehicular service caching. The notations used in this article are summarized in Table I.

### A. System Model

We consider the vehicular-MEC architectures where RSUs provide services to vehicles based on the wireless connection, as shown in Fig. 1. In vehicular-MEC architectures, the service preferences correlate with the surrounding function-features of vehicles [41]. For example, vehicles near transportation facilities are likely to request transportation management services, such as environment-aware, route-planning, etc. Therefore, the following assumptions are made.

1) Vehicles send service requests to the nearest RSUs, simplifying the network topology.
2) Considering the limited battery and resources of vehicles, the vehicular services requested by vehicles can only operate on the RSUs or cloud.
3) All vehicles have specific destinations and routes provided by electric map applications.
4) The service preferences of vehicles are decided by the surrounding function-features of vehicles.

We denote the set of RSUs as $R$ and the set of vehicles as $V$. Each RSU $r_i \in R$ possesses computation, memory, and hard-disk drive (HDD) resources to operate the vehicular services, which a tuple $r_i = (f_i, m_i, h_i)$ can characterize. In detail, $f_i$ is the computation resource that $r_i$ possesses, $m_i$ is the memory capacity of $r_i$, and $h_i$ is the HDD capacity of $r_i$. Moreover, we assume that the vehicles in this scenario have specific routes provided by electric map applications [42], which is $\text{route}_j$. $\text{route}_j$ is a vector consisting of roads vehicle $v_j \in V$ passes through, and $l_m \in \text{route}_j$ is a road. Finally, the set of the roads that are managed by RSU $r_i$ is $A_i$.

In addition, we denote the library of services as $S$, and the service request generated by vehicle $v_j$ as $t_j = (l_j, s_k)$. In detail, $l_j$ is the deadline position of request $t_j$, which is selected between the vehicle's position and destination. The RSUs need to send the service results back to the vehicle before it arrives at $l_j$. $s_k$ is the requested service of $t_j$. And the function-features of $l_j$ decide the categories of service $s_k$ [41]. Finally, the resource requirements of each vehicular service can be denoted as a tuple $s_k = (\text{cpu}_k, \text{mem}_k, \text{hard}_k)$, where $\text{cpu}_k$, $\text{mem}_k$, and $\text{hard}_k$ are the computation, memory and HDD resource requirements of $s_k$, respectively.

Furthermore, we investigate existing vehicular services [7] and classify these services into four categories: 1) entertainment; 2) smart home; 3) working; and 4) traffic management, formulating a service category set as $C = \{Ent, Home, Trans, Work\}$. The characteristics of these service categories are summarized below.

1) *Entertainment:* The entertainment services accessed in entertainment areas require high memory and HDD resources. However, their low interactions cause a low-computation resource requirement.
2) *Smart Home:* The smart-home services accessed in living areas require a high-HDD resources to store smart devices' data. In contrast, they have low-memory and computation resource requirements.
3) *Working:* The working services accessed in working areas require high computation and memory resources to support video conferences, online file editor, etc. However, they require a low-HDD resource.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

4) *Traffic Management:* The traffic-management services accessed in transportation facilities require high-computation resources for high-frequent interactions. In contrast, their memory and HDD resources requirements are low.

In addition, the distribution of these service categories also follows the surrounding function-features of vehicles. Therefore, the service preference can be calculated based on the surrounding function-features of vehicles, consisting of position of interest (POI) records [41]. Finally, we estimate the service preference of each service category $k \in C$ on road $l_m$ as

$$P_{mk}^{F} = \frac{|\mathrm{loc}_{m,k}|}{\sum_{j \in C} |\mathrm{loc}_{m,j}|} \tag{1}$$

where $|\mathrm{loc}_{m,k}|$ represents the number of POI records around the road $l_m$. The following sections will introduce the communication model and definition of network delay. Then, we formalize the problem model of the vehicular service caching.

### B. Communication Model

We assume that each RSU has the same bandwidth resources for each request. Therefore, the transmission rate between RSU $r_i$ and the vehicle at $l_m$ is

$$T_{im} = \min\left(W \cdot \log_2\left(1 + \frac{P_t}{D(l_m, r_i)^h P_n}\right), T_D\right) \tag{2}$$

where $W$ represents the wire bandwidth between RSUs and vehicles, and $P_t$ and $P_n$ is the transmission and noise power, respectively. $h$ indicates the path-loss exponent. In addition, we assume service results are transmitted when vehicles are nearest to RSUs on road $l_m$. Therefore, $D(l_m, r_i)$ is the distance between road $l_m$ and RSU $r_i$. $T_D$ is the maximum transmission rate. In vehicular network transmission, the transmission rate is also limited by the channel capacity, coding scheme, and transmission time slot, which has a theoretical maximum value [43]. Therefore, the transmission rate also has a theoretical maximum value, which the distances between vehicles and RSUs cannot influence.

Then, each request's network delay can be formalized based on the transmission rate definition. Normally, the requests' upload and computation delays are relatively small, which can be ignored. After receiving the requests, the RSUs call the service cache, generate the service results, and send the service results back to the vehicles. We assume each service result's size equals the memory requirements of its service cache. Therefore, when the requested service $s_k$ is cached on RSU $r_i$, the network delay of the service result is

$$\mathrm{delay}_{imk}^{H} = \frac{\mathrm{mem}_k}{T_{im}}. \tag{3}$$

However, because of the RSUs' limited storage and computation resources, one RSU cannot cache all services requested by vehicles, resulting in service cache misses. When the RSUs did not cache the services requested by vehicles, the RSUs will fetch the service results from the cloud and forward them to vehicles based on the wireless connection. Therefore, the network delay of the missed service is

$$\mathrm{delay}_{imk}^{M} = \frac{\mathrm{mem}_k}{T_C} + \frac{\mathrm{mem}_k}{T_{im}} \tag{4}$$

where $T_C$ is the transmission rate between RSUs and the cloud.

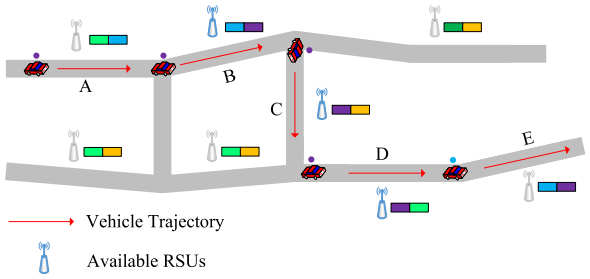



Fig. 2. Example of vehicle trajectory.

### C. Traffic Correlations Analysis

In the vehicular MEC architecture, vehicles request vehicular services with deadline positions, and RSUs must send the service result to the vehicle before the deadline positions. So, the vehicles can receive service results from the RSUs near the vehicles' routes before the deadline positions, formulating the service-sharing mechanism. The cached services can be shared among these RSUs, increasing the hit ratio of cached services. Moreover, although one RSU cannot cache all services, the service-sharing among these RSUs could increase the hit ratio. On the other hand, the vehicle trajectories formulate transportation correlations between different roads [44], [45], illustrating the cooperation among RSUs. In detail, the transportation correlations reflect the possibility distribution of vehicle mobility and the RSUs that can share cached services. Therefore, we analyze the transportation correlations of roads based on vehicle trajectories to construct the service-sharing mechanism among RSUs. By considering the transportation correlations in service caching methods, RSUs will cache the vehicular services that can maximize the possibility of service-sharing and hit ratio of cached services. Moreover, the transportation correlations should also consider the deadline position requirements of requests. When calculating the transportation correlation of road $l_m$, the requests with deadline positions before $l_m$ have been already processed. In contrast, requests with deadline positions after $l_m$ have not been processed, which can utilize the service-sharing mechanism. Therefore, the transportation correlations also focus on the directions of vehicle trajectories on each road to illustrate the possibility of service-sharing. For example, as shown in Fig. 2, the vehicle travels from road A to road E, which generates a request with a deadline position on road D. The requested service is cached on RSUs close to roads B, C, and D, formulating an available RSU set. Then, the vehicle selects the available RSU with the lowest network delay to receive the service results. And the RSU close to road C can share this service with the RSU close to D and send the service result back to vehicle from the road C. However, the service on the RSU close to road E cannot share its cached

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

service in this scenario because of violating the deadline position requirement. In conclusion, We denote the metrics of transportation correlation between road A and road B as

$$P_{AB}^{con} = \frac{\left|\{\text{route}_j | (l_A, l_B) \in \text{route}_j\}\right|}{\left|\{\text{route}_j | l_A \in \text{route}_j\}\right|}. \tag{5}$$

With the metrics of transportation correlations, we can get the possibilities of service-sharing from each RSU to the others. Based on this, we can calculate the expected profit of the service requests.

### D. Expected Profits of Service Cache

For each RSU, the expected profits of the service cache include two parts: 1) local and 2) shared expected profits. The local expected profit focuses on the requests whose deadline positions are $l_m$, which is

$$E_{imk}^{\text{local}} = P_{mk}^{F} \cdot \frac{1}{\text{delay}_{imk}^{H}}. \tag{6}$$

As shown in (6), the local expected profit is the weight reciprocal summation of network delay. With the local expected profit, the service cache method will cache services with minimum network delays and maximum service preferences. Then, we define shared expected profit as

$$E_{imk}^{\text{shared}} = \sum_{l_n \in A'_m} P_{mn}^{con} \cdot P_{mk}^{F} \cdot \frac{1}{\text{delay}_{imk}^{H}} \tag{7}$$

where $A'_m$ consists of the roads that have transportation correlations with $l_m$. Compared with the local expected profit, the shared expected profit focuses on the service-sharing with different RSUs to maximize the hit ratio of cached services. Finally, the total expected profit for service $s_k$ of RSU $r_i$ on the roads in $A_i$ is

$$E_{ik} = \sum_{l_m \in A_i} P_m^{I} \cdot \left(\alpha \cdot E_{imk}^{\text{local}} + (1-\alpha) \cdot E_{imk}^{\text{shared}}\right) \tag{8}$$

where $P_m^I$ is the number of vehicles pass through the road $l_m$, and $\alpha \in [0, 1]$ is the weight coefficient parameter representing the importance of local expected profit, which is initialized from the environmental parameters.

### E. Problem Formulization

According to the definition of transportation correlation metrics and expected profit, we formulize the vehicular service caching problem as

$$\max_{x_{ik}} \sum_{r_i \in R} \sum_{s_k \in S} x_{ik} \cdot E_{ik} \tag{9}$$

$$\text{s.t.} \sum_{s_k \in S} x_{ik} \cdot \text{cpu}_k \le f_i \tag{9c-1}$$

$$\sum_{s_k \in S} x_{ik} \cdot \text{mem}_k \le m_i \tag{9c-2}$$

$$\sum_{s_k \in S} x_{ik} \cdot \text{hard}_k \le h_i \tag{9c-3}$$

$$x_{ik} \in \{0, 1\} \tag{9c-4}$$

$$D(l_m, r_i) = \min(D(l_m, r_l)) \quad \forall r_l \in R. \tag{9c-5}$$

**Algorithm 1:** TCCS Algorithm

**input** : $R$, the set of RSUs; $S$, the library of services; $P^F$, the set of service preference; $P^{con}$, the set of transportation correlations.
**output**: *strategy*, the service caching strategy of all RSUs.

1 $strategy \leftarrow []$;
2 **for** $r_i \in R$ **do**
3 $\quad strategy^c \leftarrow []$;
4 $\quad E^c \leftarrow 0$;
5 $\quad$ **while** *iteration time* $< T$ **do**
6 $\quad\quad E^t \leftarrow 0$;
7 $\quad\quad strategy^t \leftarrow strategy^c$;
8 $\quad\quad \zeta \leftarrow randomInt()$;
9 $\quad\quad strategy^t[\zeta] \leftarrow strategy^t[\zeta] \otimes 1$;
10 $\quad\quad cpu^s, mem^s, hard^s \leftarrow calCost(strategy^t)$;
11 $\quad\quad$ **if** $cpu^s \le f_i$ *and* $mem^s \le m_i$ *and* $hard^s \le h_i$ **then**
12 $\quad\quad\quad$ **for** $s_k \in S$ **do**
13 $\quad\quad\quad\quad$ **if** $strategy^t[k] = 1$ **then**
14 $\quad\quad\quad\quad\quad$ **for** $l_m \in L_i$ **do**
15 $\quad\quad\quad\quad\quad\quad E_{ik}^{\text{local}} \leftarrow calProfit(l_m, s_k, P_{mk}^F)$;
16 $\quad\quad\quad\quad\quad\quad E_{ik}^{\text{shared}} \leftarrow calProfit(l_m, s_k, P_m^{con})$;
17 $\quad\quad\quad\quad\quad E^t += \sum \alpha \cdot E_{ik}^{\text{local}} + (1-\alpha) \cdot E_{ik}^{\text{shared}}$
18 $\quad\quad\quad$ **if** $random() > 1 - \frac{1}{1+e^{(E^t - E^c)/\varepsilon}}$ **then**
19 $\quad\quad\quad\quad E^c \leftarrow E^t$;
20 $\quad\quad\quad\quad strategy^c \leftarrow strategy^t$;
21 $\quad strategy.append(strategy^c)$;
22 **return** *strategy*

In (9), $x_{ik}$ is the decision variable. When $x_{ik} = 1$, the service $s_k$ will be cached on RSU $r_i$. Otherwise, it would not be cached there. Moreover, (9c-1)–(9c-3) represent the RSU resource capacity limitations to services caches. Constraint (9c-4) limits the value of the decision variable. Finally, in (9c-5), the vehicles on each road only select the nearest RSU as their service RSU, simplifying the network topology.

## IV. Service Caching Method Based on Transportation Correlations

This section introduces our vehicular service caching method, TCCS. As shown in (9), the vehicular service caching problem can be seen as a multidimensional knapsack problem, which is NP-hard. Therefore, heuristic algorithms are widely used to solve the vehicular service caching problem. This article uses the Gibbs sampling algorithm to search for the optimal service caching strategy. The details of TCCS algorithm are shown below.

As shown in Algorithm 1, TCCS traverses each RSU to search its optimal caching decision (line 2). First, TCCS initializes the caching decisions and profits in lines 3 and 4. Then, TCCS begins to iteratively search the caching decisions (line 5). In each iteration, TCCS randomly changes one service cache first (lines 6–9). Afterward, TCCS calculates the resource usages ($\text{cpu}^s$, $mem^s$, and $\text{hard}^s$) and drops the decision that exceeds the resource capacities (lines 10 and 11).

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

After that, the local expected profit and shared expected profit are calculated for each road (lines 15–17). Finally, in lines 18–20, the changed service caching strategy will be kept with the probability of $1-[1/(1+e^{(E^t-E^c)/\varepsilon})]$. $\varepsilon$ is the smooth parameter, which controls the exploration versus exploitation. The smaller $\varepsilon$ means the service caching decision with a lower delay will be selected. After the iteration of all RSUs, TCCS collects the service caching decisions of the RSU and output the result (lines 21 and 22).

## V. Discussion of Gibbs Sampling Algorithm

To illustrate the advantage of Gibbs sampling algorithm and TCCS, this section discusses the mechanism of Gibbs sampling algorithm. Existing vehicular service caching methods utilize heuristic algorithms to cache the vehicular services on RSUs. However, the use of most heuristic algorithms could be problematic when the number of vehicular services is massive. The heuristic algorithms usually require a large number of samples and update them based on the local and global optimal sample. Moreover, to ensure the solution's performance, more complex sample update methods are needed in existing methods, causing a high-operation time. Therefore, existing methods must reduce the iterations of heuristic algorithms to satisfy the network delay requirement of IoV environments, reducing the performance of cached services.

Compared with other heuristic algorithms, Gibbs sampling algorithm is space and time efficient, satisfying the network delay requirement of IoV environments. Moreover, the surrounding function-features of vehicles follow the heavy tail distribution, which means the service preferences of areas are similar. Therefore, in the vehicular service caching problem, Gibbs sampling algorithm could use the service cache strategy of the nearby RSU as the initial sample and update the initial sample based on the expected profit. Based on this, Gibbs only needs to update a small part of the initial sample, increasing the possibility of finding the optimal strategy.

To illustrate the advantage of Gibbs sampling algorithm, we take Fig. 2 as an example. We assume that there is a business district with multiple places of entertainment on the roads within Fig. 2. The vehicles traveling on these roads are likely to request entertainment services, causing similar service preferences on these roads. Therefore, the RSU should cache the entertainment services first to satisfy the requests on these roads, and the other services should be cached for service-sharing. When Gibbs sampling algorithm obtains the cache strategy of the RSU near road A, the RSU near road B can utilize this cache strategy as an initial sample and update it to enlarge the profits of service-sharing. However, the other heuristic algorithms require more than one sample, limiting the effect of nearby RSUs' cache strategy.

## VI. Dynamic Service Cache Replacement Framework

In urban areas, the vehicles' requests for service are time-varying. Specifically, the mobility of vehicles changes the transportation correlations, influencing the utilization of cached services. Therefore, the service cache replacement mechanism should consider the changed transportation correlations and reconstruct the service-sharing mechanism. To ensure the utilization of service caches, we adopt a dynamic service cache replacement framework, as shown in Fig. 3. In our framework, the service cache replacement is performed at the end of each time frame. By analyzing the transportation correlations in each time frame, the expected profits of cached services are evaluated, and the infrequently requested services are replaced with the services with maximum expected profits. We denote the set of time frames as $\Gamma$. In time frame $\tau \in \Gamma$, the RSUs collect the vehicle trajectories $traj_\tau$ and requests $req_\tau$. $\text{route}_j^\tau \in traj_\tau$ is the trajectory of vehicle $v_j$ till $\tau$, and $hit_{qki}^\tau \in req_\tau$ represents if the requested service $s_k$ is cached on RSU $r_i$. When $hit_{qki}^\tau = 1$, the service $s_k$ is cached on $r_i$ in time frame $\tau$, which is a hit request. Otherwise, when $hit_{qki}^\tau = 0$, the RSUs did not cache the service $s_k$ in time frame $\tau$. With vehicle trajectories, we can calculate the transportation correlations within time frame $\tau$. The transportation correlation between roads $l_a$ and $l_b$ is

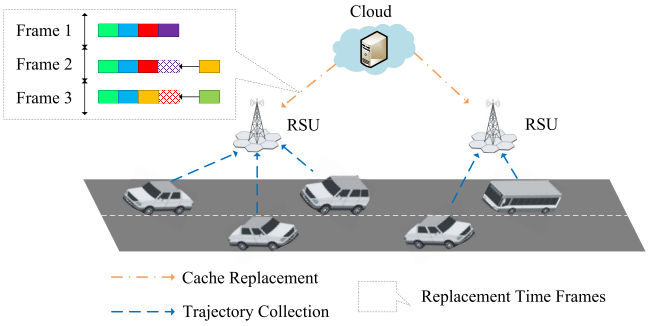



Fig. 3. Dynamic service cache replacement framework.

$$P_{ab\tau}^{con} = \frac{\left|\{\text{route}_j^\tau | (l_a, l_b) \in \text{route}_j^\tau\}\right|}{\left|\{\text{route}_j^\tau | l_a \in \text{route}_j^\tau\}\right|}. \tag{10}$$

Then, the hit ratios of services can be calculated based on the requests set $req_\tau$. In detail, the hit ratio of service $s_k$ is the ratio between the request number of $s_k$ and the total request number, which is

$$P_{ik\tau}^{hit} = \frac{\sum_{q=1}^{|req_\tau|} hit_{qki}^\tau}{|req_\tau|}. \tag{11}$$

Our service cache replacement framework aims to improve the hit ratio of cached services. In detail, the cached services with low-hit ratio should be removed to release the RSUs' resources. Then, the uncached frequently requested services should be cached to maximize their hit ratio by utilizing the released resources. The following sections introduce which services should be removed and cached. Then, we give the metrics of each service cache replacement profit. Finally, we formalize the problem model of vehicular service cache replacement.

### A. Recognization of Services Status

Our service cache replacement framework sets the status of services on each RSU based on the number of hit requests,

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

including frequently requested and infrequently requested. We introduce a threshold $\theta$ to classify the service statuses. When the number of hit requests $|hit^{\tau}_{qki}|$ is higher than $\theta$, at least $\theta$ number of vehicles request for the service $s_k$ on RSU $r_i$, meaning a frequently requested status. Otherwise, the status of $s_k$ is infrequently requested. The detailed definition of service status is

$$e^{\tau}_{ik} = \begin{cases} 1, & \left|hit^{\tau}_{qki}\right| \geq \theta \\ 0, & \left|hit^{\tau}_{qki}\right| < \theta. \end{cases} \tag{12}$$

After the recognization of service status, our service cache replacement framework removes the infrequently requested services to release the resources of RSUs. Then, the framework needs to evaluate all uncached services and cache the services with the highest profits. We will define the profit of service cache replacement in the following content.

### B. Profits Service Cache Replacement

Like vehicular service caching method, the services cached on one RSU can be shared with others in the service cache replacement framework. Therefore, the profits of service cache replacement should also consider the transportation correlations. In detail, we define the profits of service cache replacement as the reciprocal of expected profit, including local and shared expected profits. Moreover, we replace the service preference with the hit ratio. The detailed profit of service cache replacement for RSU $r_i$ is

$$E^{\tau}_{ik} = \alpha \sum_{l_m \in L_i} P^{hit}_{mk\tau} \frac{1}{\text{delay}^{h}_{ik}} + (1-\alpha) \sum_{l_m \in L_i} \sum_{l_n \in A'_m} P^{con}_{mn\tau} P^{hit}_{nk\tau} \frac{1}{\text{delay}^{h}_{ik}}. \tag{13}$$

As shown in (13), higher hit ratio means higher profits. And the transportation correlation used in this equation is calculated based on the vehicle trajectories within the time frame $\tau$. With the definition of replacement profit, the services with higher hit ratio and more transportation correlation will be cached, increasing the hit ratio of cached services.

### C. Problem Formulization

Like vehicular service caching method, the service cache replacement framework needs to replace infrequently requested services with uncached services to maximize profits and satisfy the constraints of released resources. Therefore, we give the calculation of released resources, which is

$$f^{\tau}_{i} = f_i - \sum_{s_k \in S} x^{\tau-1}_{ik} e^{\tau}_{ik} \text{cpu}_k \tag{14}$$

$$m^{\tau}_{i} = m_i - \sum_{s_k \in S} x^{\tau-1}_{ik} e^{\tau}_{ik} \text{mem}_k \tag{15}$$

$$h^{\tau}_{i} = h_i - \sum_{s_k \in S} x^{\tau-1}_{ik} e^{\tau}_{ik} \text{hard}_k \tag{16}$$

where $x^{\tau-1}_{ik}$ represents the cache decision in the time frame $\tau-1$. When $\tau = 1$, $x^{\tau-1}_{ik}$ equals $x_{ik}$, meaning the service cache strategies generated by TCCS. Afterward, the problem model of service cache replacement in time frame $\tau$ is

$$\max_{x^{\tau}_{ik}} \sum_{r_i \in R} \sum_{s_k \in S} x^{\tau}_{ik} \cdot E^{\tau}_{ik} \tag{17}$$

$$\text{s.t.} \sum_{s_k \in S} x^{\tau}_{ik} \cdot \text{cpu}_k \leq f^{\tau}_{i} \tag{17c-1}$$

$$\sum_{s_k \in S} x^{\tau}_{ik} \cdot \text{mem}_k \leq m^{\tau}_{i} \tag{17c-2}$$

$$\sum_{s_k \in S} x^{\tau}_{ik} \cdot \text{hard}_k \leq h^{\tau}_{i} \tag{17c-3}$$

$$x^{\tau}_{ik} \in \{0, 1\} \tag{17c-4}$$

$$e^{\tau}_{ik} \otimes x^{\tau-1}_{ik} = 1 \quad \forall s_k \in S. \tag{17c-5}$$

As shown in (17), the service cache replacement aims to maximize the expected profits while satisfying the resource constraints. In detail, (17c-1)–(17c-3) represent the released resource limitations to services cache replacement. Constraint (17c-4) represents the value of $x^{\tau}_{ik}$ is $\{0, 1\}$. Finally, (17c-5) illustrates the candidate service list in this problem. In detail, all frequently requested services should be considered in the service replacement problem. Moreover, when released resources are enough for all frequently requested services, part of the infrequently requested services could be remained in RSUs. Therefore, the infrequently requested service should also be considered in this problem.

## VII. Dynamic Service Cache Replacement Method Based on Transportation Correlations

This section introduces our dynamic service cache replacement method, DCRS. Like service caching method, the dynamic service replacement problem can also be seen as a multidimensional knapsack problem. However, our service cache replacement framework only focuses on the infrequently requested and uncached services, decreasing the solution space of the service cache replacement problem. Therefore, the existing integer linear programming (ILP) solver could generate the optimal replacement decision with low-processing time. And we use the ILP solver in DCRS to search for the replacement decision of each RSU.

In addition, our methods are verified in a simulated vehicular-MEC environment with a finite simulation time. Therefore, when simulation time becomes longer, the requests' deadline positions are close to the vehicles' positions, decreasing the number of available RSUs. It further reduces the possibility of service-sharing. To address this problem, we apply a profit adjustment mechanism in DCRS, ignoring the shared expected profit after a long simulation time. The detailed procedure of DCRS is shown in Algorithm 2.

As shown in Algorithm 2, DCRS first sets the weight coefficient $\alpha^{\tau}$ in lines 1–4. When $\tau$ is higher than $\Psi$, most vehicles are near their destination. DCRS sets $\alpha^{\tau}$ as 1, meaning the ignorance of shared expected profits. Otherwise, the $\alpha^{\tau}$ equals to $\alpha$. After that, DCRS formulates the problem model of service cache replacement for each RSU (lines 5–20). In detail, DCRS calculates the hit ratio of each service and

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 2:** DCRS Algorithm

**input** : $R$, the set of RSUs; $S$, the library of services; $req_\tau$, the set of vehicle requests in time frame $\tau$; $traj_\tau$, the set of vehicle trajectories in time frame $\tau$; $x^{\tau-1}$, the cache decisions in time frame $\tau-1$; $\tau$, the time frame; $\alpha$, the profit weight coefficient.

**output**: $strategy^\tau$, the service caching strategy of all RSUs in time frame $\tau$.

1 **if** $\tau \leq \Psi$ **then**
2 $\quad \alpha^\tau = \alpha$
3 **else**
4 $\quad \alpha^\tau = 1$
5 $strategy^\tau \leftarrow []$;
6 $P_\tau^{con} \leftarrow calCorrelations(traj_\tau)$;
7 **for** $r_i \in R$ **do**
8 $\quad releasedCache \leftarrow []$;
9 $\quad P_\tau^{hit} \leftarrow []$;
10 $\quad$ **for** $s_k \in S$ **do**
11 $\quad\quad P_\tau^{hit}$.append$(P_{mk\tau}^{hit})$;
12 $\quad\quad$ **if** $\sum_{l_m \in L_i} \left| hit_{qkm}^\tau \right| < \theta$ **then**
13 $\quad\quad\quad releasedCache$.append$(s_k)$;
14 $\quad$ **if** $releasedCache = []$ **then**
15 $\quad\quad$ continue;
16 $\quad f_i^\tau, m_i^\tau, h_i^\tau \leftarrow sumSource(releasedCache)$;
17 $\quad cand \leftarrow releasedCache$;
18 $\quad$ **for** $s_k \in S$ **do**
19 $\quad\quad$ **if** $x_{ik}^{\tau-1} = 0 \cap e_{ik}^\tau = 0$ **then**
20 $\quad\quad\quad cand$.append$(s_k)$;
21 $\quad strategy_i^\tau \leftarrow solver(cand, P_\tau^{hit}, P_\tau^{con}, f_i^\tau, m_i^\tau, h_i^\tau)$;
22 $\quad strategy^\tau$.append$(strategy_i^\tau)$;
23 **return** $strategy^\tau$

finds the infrequently requested services in lines 5–13. Then, DCRS checks the number of infrequently requested services. If the RSU has no infrequently requested services, DCRS will continue searching for the replacement decision for the next RSU (lines 14 and 15). Afterward, the released resources are calculated, and the candidate service list is initialized in lines 16 and 17. Moreover, all frequently requested services not cached by $r_i$ are appended to the candidate service list in lines 18–20. Finally, DCRS calls the ILP solver to generate the optimal replacement decision and generates the service cache replacement strategy for all RSUs (lines 21–23).

TCCS traverses all RSUs first and iteratively searches for the optimal service caching strategy. Then, for each iteration, TCCS calculates each road's local and shared expected profits. Assuming the iteration time, the size of RSU set, and road set are $T$, $N$, and $M$, the time complexity of TCCS is $o(TNM)$. The time complexity of DCRS depends on the number of RSUs and services. In addition, the ILP solver can be seen as a divide-and-conquer algorithm, dividing the whole ILP into several subproblems. Assuming the number of RSUs and candidate services are $N$ and $K$, the time complexity of DCRS is $o(NK \log K)$. Compared with TCCS and DCRS, most of the existing vehicular service caching methods need to traverse all services and calculate the service preferences [5], [14], [19], [22]. Therefore, we assume the iteration time, the size of RSU set, road set, and service number are $T$, $N$, $M$, and $K$. The time complexity of these vehicular service caching methods is $o(TNMK)$, resulting in a higher time consumption than our methods.

## VIII. Performance Evaluation

### A. Data Set

We evaluate TCCS, DCRS, and other existing baselines in public vehicle trajectory data sets of Shenzhen, China [46]. We implement a vehicular-MEC architecture with real vehicle trajectories in Veins 5.1, including SUMO 1.8.0, OMNeT++ 5.8.2, and Python 3.8.10, to obtain performance close to the real world. In addition, the ILP solver used in the simulated vehicular-MEC architecture is *CBC 2.10.3*. Veins integrates the road environment and network property, where vehicles and RSUs can access all road and network information. Moreover, compared with existing vehicular network simulators, Veins simulates the transmission process of each network message and the vehicular MEC environment with more network details, which is closer to the real world. However, raw vehicle trajectory data sets are not supported by Veins. It requires the vehicle trajectories to match a specific map environment. Therefore, we apply the map-match algorithm to map the vehicle trajectories to the road environment. Moreover, Veins only implements a basic traffic event broadcasting protocol that does not support computation offloading simulation. To address this problem, we realize the vehicular MEC architecture in Veins application layers of vehicles and RSUs. Finally, the vehicles in Veins select the RSU with minimum distance to receive the service results. We evaluate the hit ratio and network delay based on the request records of Veins.

The vehicle trajectory data set of Shenzhen, China includes 19542 vehicle trajectories in 15 days. In addition, each vehicle trajectory consists of its trace points' longitude, latitude, and timestamp. The RSU number in Shenzhen, China is 63. Moreover, the map size of Shenzhen, China is 9km × 7km. In detail, we use 850 vehicle trajectories on 1 January 2016, to compare the performance of service cache methods. Moreover, we also use 842 vehicle trajectories on 2 January 2016, to verify the performance of the service cache replacement framework. The service requests are generated based on POI records of AutoNavi map. In addition, we investigate existing research on vehicular service caching to set the parameters of RSUs and vehicles. The detailed parameters of our simulation environment are shown in Table II.

In addition, we calculate the distances between roads and RSUs, whose values are above 6 m. Therefore, we set the value of theoretic maximum transmission rate based on the minimum distance value [47]. Finally, the theoretic maximum transmission rate is 60 Mb/s. Then, the value of $T_C$ is decided by the network interface's transmission rate of the cloud. Moreover, the cloud has numerous services with different network resource requirements. Therefore, we assume the total transmission rate of the cloud and RSUs is 1 Gb/s, meaning the transmission rate of the cloud and each RSU is 16 Mb/s [48].

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

TABLE II
EXPERIMENT PARAMETERS

| Simulation Parameters | Values |
|---|---|
| RSU computational capability (GHz) | $[10, 20]$ |
| RSU memory capacity (GB) | $[16, 20]$ |
| RSU HDD capacity (GB) | 600 |
| RSU coverage (m) | 1000 |
| Computation resources requirement of services (GHz) | $[0.8, 4.5]$ |
| Memory resources requirement of services (MB) | $[20, 100]$ |
| HDD resources requirement of services (GB) | $[40, 130]$ |
| The service number of each service categoriy | 5 |
| Number of request generated by a vehicle | $[1, 5]$ |
| Communication transmission power (mW) | 250 |
| Communication noise power (mW) | 0.005 |
| Communication bandwidth (MHZ) | 10 |
| Maximum transmission rate (Mbps) | 60 |
| Transmission rate between RSUs and the cloud (Mbps) | 16 |
| Simulation time (s) | 2000 |
| Time frame size (s) | 60 |
| Profit adjustment threshold $\Psi$ (s) | 360 |
| Weight coefficient $\alpha$ | 0.7 |

### B. Baseline Algorithms

In our experiment, we implement the following baselines in our simulation environment and compare them with our methods.

*GreedyStrategy [14]:* GreedyStrategy caches the services based on the expected profits. First, GreedyStrategy calculates the expected profit for each road based on its service preference. Then, it sorts the expected profits and selects the services with maximum expected profits.

*PopularityBased [22]:* PopularityBased randomly caches the services based on the probability density of POI. PopularityBased uses a roulette selection method to select the service categories and randomly cache the specific services of selected service categories.

*MobilityAware [19]:* MobilityAware uses a semi-Markov process to model vehicle mobility and predict vehicle positions. Then, the prediction results formulate the transition matrix of RSUs, indicating the possibility of vehicle mobility among RSUs. In addition, MobilityAware finds 2 RSUs with maximum transition values for each RSU to construct cooperation groups where the cached services are shared. Finally, the vehicular services with maximum preferences in the cooperation group are cached by MobilityAware. Moreover, the cached services are replaced based on the hit ratio.

*GroupBased [5]:* GroupBased groups the RSUs into 9 groups, each with 8 RSUs. Then, GroupBased divides the resources into group and global resources with ratios of 0.7 and 0.3. Afterward, GroupBased caches the services with the group resources based on the service preferences within groups. In addition, GroupBased randomly caches the services with global resources to satisfy the requests for services with low preferences. Finally, GroupBased also applies a service cache replacement to replace the services with low-hit ratios with the frequently requested services.

### C. Comparison of Total Hit Ratio

This section compares the total hit ratio of the service caching methods on 1 January 2016. The detailed total hit ratio of service caching methods is shown in Fig. 4.

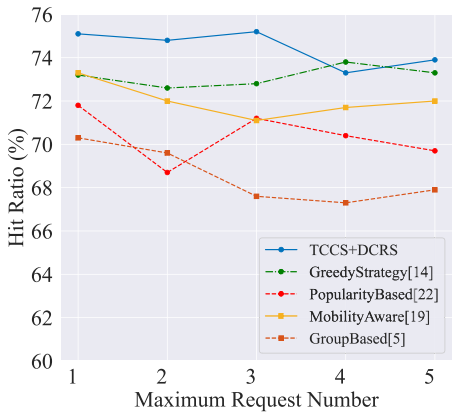



Fig. 4. Total hit ratio of service caching methods on 1 January 2016.

As shown in Fig. 4, TCCS and DCRS possess the highest total hit ratio among these service caching methods. It means that our methods effectively estimate the service preference of vehicles and cache the frequently requested services. Moreover, the service-sharing among RSUs also increases the total hit ratio. However, with the increment of the maximum request number, more requests' deadline positions are close to the vehicles' destinations, decreasing the possibility of service-sharing. Therefore, the total hit ratio of TCCS and DCRS decreases with the increment of the maximum request number.

In addition, GreedyStrategy considers both local and shared expected profit, causing a total hit ratio just lower than TCCS and DCRS. It means that only caching the services with the highest expected profits cannot ensure the total hit ratio of cached services. Nevertheless, GreedyStrategy ignores the service cache replacement, decreasing its total hit ratio. Moreover, MobilityAware has a close total hit ratio with GreedyStrategy. It means that the cooperation group mechanism of MobilityAware increases the possibility of service sharing. However, MobilityAware lacks the consideration of service sharing possibility, decreasing its total hit ratio.

Moreover, PopularityBased has a total hit ratio lower than GreedyStrategy. It is because PopularityBased only focuses on the service preference of a single RSU, ignoring the service-sharing. Finally, the total hit ratio of GroupBased is the lowest. GroupBased focuses more on cooperation among RSUs, influencing the satisfaction of local service preference.

Furthermore, we also compare the total hit ratio of service caching methods on 2 January 2016 to verify the performance of service cache replacement. The detailed results are shown in Fig. 5.

As shown in Fig. 5, TCCS and DCRS have a similar total hit ratio to that on 1 January 2016. It means that DCRS effectively ensures the total hit ratio of cached services by replacing the infrequently requested services with frequently requested services. Moreover, the higher maximum request number decreases the number of available RSUs, decreasing the total hit ratio. The total hit ratio of GreedyStrategy is also similar to that on 1 January 2016. It is because the transportation correlations of these two days are similar, guaranteeing the accuracy of profit expectations. However, the simple searching strategy of GreedyStrategy also decreases its total hit ratio.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

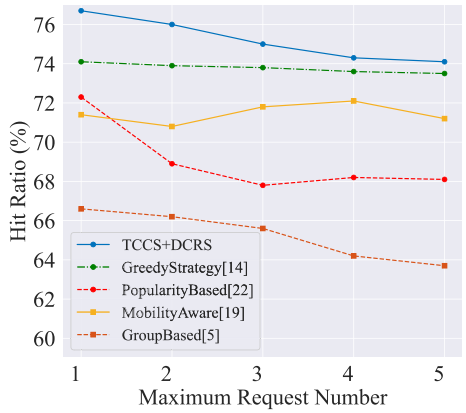



Fig. 5. Total hit ratio of service caching methods on 2 January 2016.

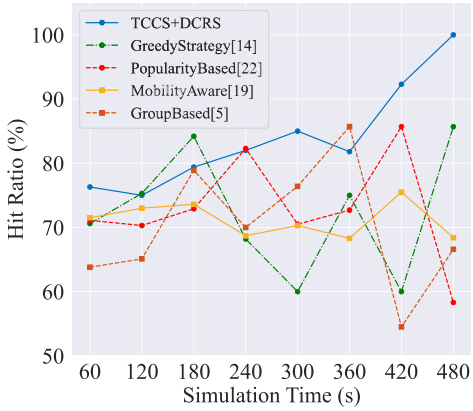



Fig. 6. Total hit ratio of service caching methods in each time frame on 1 January 2016.

Nevertheless, the total hit ratios of PopularityBased and GroupBased are obviously decreased. First, PopularityBased ignores the service cache replacement, resulting in a decrement in the total hit ratio. Moreover, GroupBased only considers the cooperation of RSUs in each group, which also decreases the total hit ratio. In addition, the total hit ratio of MobilityAware is just lower than GreedyStrategy. It is because the cooperation groups of MobilityAware are constructed based on vehicle mobility, which is similar in these two days. However, lacking the consideration of mobility changes in service cache replacement, MobilityAware has a lower total hit ratio than GreedyStrategy. Finally, the total hit ratio of TCCS and DCRS is higher than GroupBased by 15.26%.

In addition, to verify the effect profit adjustment mechanism in DCRS, we also calculate the total hit ratio of service caching methods in each time frame. In the vehicle trajectory data set used in this article, most vehicles reach their destinations before 500 s. The number of requests generated after 500 s is too low to show the accurate total hit ratio of service caching methods. Therefore, Fig. 6 only illustrates the total hit ratio of the service caching methods before 480 s.

As shown in Fig. 6, the total hit ratio of TCCS and DCRS continuously increases with the simulation time. It means that DCRS effectively replaces the infrequently requested services with frequently requested services, ensuring the total hit ratio of cached services. Moreover, when the simulation time is higher than $\Psi$, DCRS only focuses on the local expected profit, resulting in a decrement in the total hit ratio at 360 s. After 360 s, because of the effect profit adjustment mechanism in DCRS, the total hit ratio of TCCS and DCRS continue to increase with simulation time. However, lacking the service cache replacement mechanism, the total hit ratios of PopularityBased and GreedyStrategy randomly change with the simulation time. In addition, the service cache replacement mechanism of GroupBased and MobilityAware has a low effect on the total hit ratio. It is because GroupBased and MobilityAware ignores the changes in cooperation among RSUs caused by vehicle mobility.

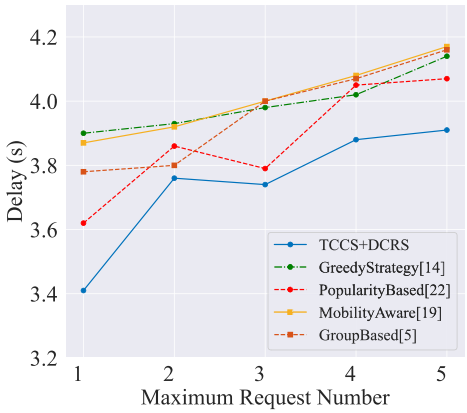



Fig. 7. Average network delay of service caching methods on 1 January 2016.

### D. Comparison of Average Network Delay

In real vehicular-MEC architecture, the network delay affects the performance of vehicular services. Therefore, this section compares the average network delay of the service caching methods. The detailed results on 1 January 2016 are shown in Fig. 7.

As shown in Fig. 7, TCCS and DCRS have the minimum average network delay among these service caching methods because of high-total hit ratio. Moreover, the service-sharing mechanism also optimizes the average network delay of TCCS and DCRS, where vehicles select the RSUs with the minimum distances to send back the service results. Compared with TCCS and DCRS, lacking the service cache replacement mechanism, GreedyStrategy has an average network delay higher than TCCS and DCRS. Vehicle mobility changes transportation correlations, influencing the service-sharing. In addition, with the increment of the maximum request number, the average network delays of TCCS, DCRS, and GreedyStrategy become similar. It means the number of available RSUs also influences the average network delay of service caching methods. Moreover, the average network delay of MobilityAware is close to GreedyStrategy. It is because MobilityAware ignores the network delay in the service caching process.

Furthermore, the average network delay of PopularityBased is between TCCS, DCRS, and GreedyStrategy. It is because

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

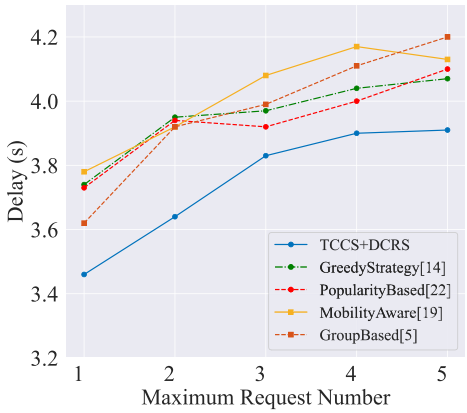



Fig. 8. Average network delay of service caching methods on 2 January 2016.

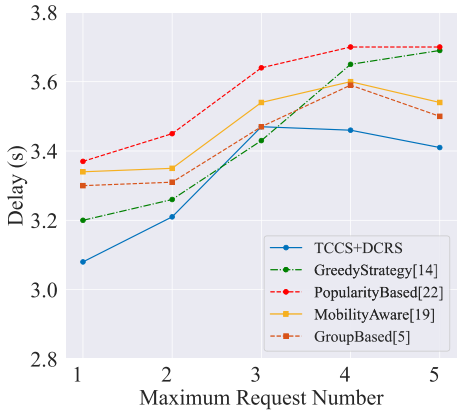



Fig. 9. Average network delay of hit requests on 1 January 2016.

PopularityBased caches each road's most popular service cached on its closest RSU, resulting in a low-average network delay. However, the low-total hit ratio of PopularityBased also decreases its average network delay. Finally, GroupBased has the highest average network delay among these service caching methods. It means the cooperation in each group increases the distance between vehicles and RSUs that cached requested services, causing a low-transmission rate of service results.

In addition, we also compare the average network delay on 2 January 2016 to verify the effect of the service cache replacement mechanism. The detailed results are shown in Fig. 8.

As shown in Fig. 8, the average network delay of TCCS and DCRS is similar to that on 1 January 2016. It means the service cache replacement mechanism in DCRS also optimizes the average network delay. However, the average network delay of GreedyStrategy increases on 2 January 2016. It is because the simple searching strategy of GreedyStrategy limits the service-sharing. In addition, similar to results on 1 January 2016, the increment of the maximum request number also decreases the number of available RSUs, increasing the average network delay. Like the results on 1 January 2016, MobilityAware has the almost highest average network delay. It means that MobilityAware ignores the optimization of network delay. Moreover, the average network delays of PopularityBased and GroupBased are higher than that on 1 January 2016. It means the change in vehicle mobility increases the average network delay of these two algorithms. Finally, the average network delay of TCCS and DCRS is lower than GroupBased by 5.6%.

We also calculate the average network delay of hit requests to verify the performance of service-sharing mechanism. The detailed results are shown in Fig. 9.

The result in Fig. 9 illustrates that TCCS and DCRS's average network delay of hit requests is the lowest. The consideration of shared expected profit effectively enlarges the effect of service-sharing mechanism. Moreover, the service cache replacement mechanism also optimizes the network delay. GreedyStrategy's average network delay of hit requests is higher than TCCS and DCRS. It means that the simple search strategy of GreedyStrategy influences the service-sharing and increases the network delay of cached services. Then, because of the inefficient cooperation of each group, GroupBased's average network delay of hit requests is higher than TCCS, DCRS, and GreedyStrategy. The cooperation within groups cannot cache the services with the minimum network delay. MobilityAware's average network delay of hit requests is close to GroupBased. It is because MobilityAware ignores network delay in service caching. Finally, PopularityBased's average network delay of hit requests is the highest. Ignoring the cooperation of RSUs, PopularityBased cannot ensure network delay.

## IX. Conclusion

This article studied the vehicular service caching problem with the aim of optimizing the hit ratio and network delay of cached services. First, we use the surrounding function-features of vehicles to estimate the service preference of vehicles. Then, we propose two vehicular service caching methods, TCCS and DCRS, focusing on network delay, hit ratio, and changes in cooperation among RSUs caused by vehicle mobility. Simulation experiments are conducted in a real-world data set. Our proposed methods outperform the baseline methods in hit ratio and network delay by 15.26% and 5.6%, respectively. Moreover, our methods minimize the negative influence on service caching caused by vehicle mobility. One promising future direction is to optimize energy consumption by considering the allocation of wireless resources.

## References

[1] J. E. Siegel, D. C. Erb, and S. E. Sarma, "A survey of the connected vehicle landscape—Architectures, enabling technologies, applications, and development areas," *IEEE Trans. Intell. Transp. Syst.*, vol. 19, no. 8, pp. 2391–2406, Aug. 2018.

[2] S. Zhou, P. P. Netalkar, Y. Chang, Y. Xu, and J. Chao, "The MEC-based architecture design for low-latency and fast hand-off vehicular networking," in *Proc. IEEE 88th Veh. Technol. Conf. (VTC-Fall)*, Aug. 2018, pp. 1–7.

[3] H. Materwala, L. Ismail, R. M. Shubair, and R. Buyya, "Energy-SLA-aware genetic algorithm for edge–cloud integrated computation offloading in vehicular networks," *Future Gener. Comput. Syst.*, vol. 135, pp. 205–222, Oct. 2022.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.

[4] R. Yadav, W. Zhang, O. Kaiwartya, H. Song, and S. Yu, "Energy-latency tradeoff for dynamic computation offloading in vehicular fog computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14198–14211, Dec. 2020.

[5] F. Zeng, K. Zhang, L. Wu, and J. Wu, "Efficient caching in vehicular edge computing based on edge-cloud collaboration," *IEEE Trans. Veh. Technol.*, vol. 72, no. 2, pp. 2468–2481, Feb. 2023.

[6] Y. Li, B. Yang, H. Wu, Q. Han, C. Chen, and X. Guan, "Joint offloading decision and resource allocation for vehicular fog-edge computing networks: A contract-Stackelberg approach," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15969–15982, Sep. 2022.

[7] H. Hu, Y. Zheng, Z. Bao, G. Li, J. Feng, and R. Cheng, "Crowdsourced POI Labelling: Location-aware result inference and task assignment," in *Proc. IEEE 32nd Int. Conf. Data Eng. (ICDE)*, May 2016, pp. 61–72.

[8] Q. Shen, B.-J. Hu, and E. Xia, "Dependency-aware task offloading and service caching in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13182–13197, Dec. 2022.

[9] D. Zeng, J. Zhang, L. Gu, S. Guo, and J. Luo, "Energy-efficient coordinated multipoint scheduling in green cloud radio access network," *IEEE Trans. Veh. Technol.*, vol. 67, no. 10, pp. 9922–9930, Oct. 2018.

[10] C. Song, W. Xu, T. Wu, S. Yu, P. Zeng, and N. Zhang, "QoE-driven edge caching in vehicle networks based on deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 5286–5295, Jun. 2021.

[11] A. Mondal, S. Misra, and G. Das, "CALM: QoS-aware vehicular sensor-as-a-service provisioning in cache-enabled multi-sensor cloud," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 3, pp. 1564–1573, Sep. 2022.

[12] A. Malik, J. Kim, K. S. Kim, and W.-Y. Shin, "A personalized preference learning framework for caching in mobile networks," *IEEE Trans. Mobile Comput.*, vol. 20, no. 6, pp. 2124–2139, Jun. 2021.

[13] Z. Li, C. Yang, X. Huang, W. Zeng, and S. Xie, "CoOR: Collaborative task offloading and service caching replacement for vehicular edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 7, pp. 9676–9681, Jul. 2023.

[14] B. Li and R. Wu, "Joint perception data caching and computation offloading in MEC-enabled vehicular networks," *Comput. Commun.*, vol. 199, pp. 139–152, Feb. 2023.

[15] Y. Zhang, H. Zhang, and K. Long, "Energy efficient resource allocation in cache based Terahertz vehicular networks: A mean-field game approach," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 5275–5285, Jun. 2021.

[16] A. Mahmood, C. E. Casetti, C. F. Chiasserini, P. Giaccone, and J. Härri, "The RICH Prefetching in edge caches for in-order delivery to connected cars," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 4–18, Jan. 2019.

[17] Y. Kyung and T. Song, "CSV: Content service offloading system with vehicular caching," *Sensors*, vol. 22, no. 20, p. 7967, Jan. 2022.

[18] Z. Zhang, C.-H. Lung, M. St-Hilaire, and I. Lambadaris, "Smart proactive caching: Empower the video delivery for autonomous vehicles in ICN-based networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7955–7965, Jul. 2020.

[19] G. Yu, Y. He, J. Wu, Z. Chen, and J. Pan, "Mobility-aware proactive edge caching for large files in the Internet of Vehicles," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11293–11305, Jul. 2023.

[20] Z. Yu, J. Hu, G. Min, Z. Zhao, W. Miao, and M. S. Hossain, "Mobility-aware proactive edge caching for connected vehicles using federated learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 5341–5351, Aug. 2021.

[21] R. Wu, G. Tang, T. Chen, D. Guo, L. Luo, and W. Kang, "A profit-aware coalition game for cooperative content caching at the network edge," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 1361–1373, Jan. 2022.

[22] T.-V. Nguyen et al., "User-preference-based proactive caching in edge networks," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2021, pp. 755–757.

[23] R. Aghazadeh, A. Shahidinejad, and M. Ghobaei-Arani, "Proactive content caching in edge computing environment: A review," *Softw. Pract. Exp.*, vol. 53, no. 3, pp. 811–855, 2023.

[24] J. Du et al., "MADDPG-based joint service placement and task offloading in MEC empowered air-ground integrated networks," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10600–10615, Mar. 2024.

[25] C.-L. Wu, T.-C. Chiu, C.-Y. Wang, and A.-C. Pang, "Mobility-aware deep reinforcement learning with glimpse mobility prediction in edge computing," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–7.

[26] X. Huang, L. He, and W. Zhang, "Vehicle speed aware computing task offloading and resource allocation based on multi-agent reinforcement learning in a vehicular edge computing network," in *Proc. IEEE Int. Conf. Edge Comput. (EDGE)*, Oct. 2020, pp. 1–8.

[27] Y. AlNagar, R. H. Gohary, S. Hosny, and A. A. El-Sherif, "Mobility-aware edge caching for minimizing latency in vehicular networks," *IEEE Open J. Veh. Technol.*, vol. 3, pp. 68–84, 2022.

[28] X. Zhang, H. Zhong, C. Fan, I. Bolodurina, and J. Cui, "CBACS: A privacy-preserving and efficient cache-based access control scheme for software defined vehicular networks," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1930–1945, 2022.

[29] L. Gu, D. Zeng, W. Li, S. Guo, A. Y. Zomaya, and H. Jin, "Intelligent VNF orchestration and flow scheduling via model-assisted deep reinforcement learning," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 2, pp. 279–291, Feb. 2020.

[30] I. A. Elgendy, W.-Z. Zhang, H. He, B. B. Gupta, and A. A. Abd El-Latif, "Joint computation offloading and task caching for multi-user and multi-task MEC systems: Reinforcement learning-based algorithms," *Wireless Netw.*, vol. 27, pp. 2023–2038, Feb. 2021.

[31] M. Choi, M. Shin, and J. Kim, "Dynamic video delivery using deep reinforcement learning for device-to-device underlaid cache-enabled Internet of Vehicle networks," *J. Commun. Netw.*, vol. 23, no. 2, pp. 117–128, Apr. 2021.

[32] T. Liu, S. Ni, X. Li, Y. Zhu, L. Kong, and Y. Yang, "Deep reinforcement learning based approach for online service placement and computation resource allocation in edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3870–3881, Jul. 2023.

[33] L. Gu, D. Zeng, J. Hu, B. Li, and H. Jin, "Layer aware microservice placement and request scheduling at the edge," in *Proc. IEEE Conf. Comput. Commun.*, May 2021, pp. 1–9.

[34] D. Ren, X. Gui, and K. Zhang, "Adaptive request scheduling and service caching for MEC-assisted IoT networks: An online learning approach," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 17372–17386, Sep. 2022.

[35] L. Gu, D. Zeng, S. Guo, and B. Ye, "Leverage parking cars in a two-tier data center," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Apr. 2013, pp. 4665–4670.

[36] J. Du et al., "Resource pricing and allocation in MEC enabled blockchain systems: An A3C deep reinforcement learning approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 1, pp. 33–44, Jan./Feb. 2022.

[37] T. Zong, C. Li, Y. Lei, G. Li, H. Cao, and Y. Liu, "Cocktail edge caching: Ride dynamic trends of content popularity with ensemble learning," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 208–219, Feb. 2023.

[38] X. Dai et al., "Task offloading for cloud-assisted fog computing with dynamic service caching in Enterprise management systems," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 662–672, Jan. 2023.

[39] Z. Degan, W. Shuo, Z. Jie, Z. Haoli, Z. Ting, and Z. Xiumei, "A content distribution method of Internet of Vehicles based on edge cache and immune cloning strategy," *Ad Hoc Netw.*, vol. 138, Jan. 2023, Art. no. 103012.

[40] L. Gu et al., "Fairness-aware dynamic rate control and flow scheduling for network utility Maximization in network service chain," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1059–1071, May 2019.

[41] Y. Gao, Z. Duan, W. Shi, J. Feng, and Y.-Y. Chiang, "Personalized recommendation method of POI based on deep neural network," in *Proc. 6th Int. Conf. Behav. Econ. Socio-Cultural Comput. (BESC)*, Oct. 2019, pp. 1–6.

[42] I. Skog and P. Handel, "In-car positioning and navigation technologies—A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 10, no. 1, pp. 4–21, Mar. 2009.

[43] M. Zorzi and A. Chockalingam, "Wireless communications," in *Encyclopedia of Physical Science and Technology (Third Edition)*, R. A. Meyers, Ed. New York, NY, USA: Academic Press, Jan. 2003, pp. 851–874.

[44] H. Chen, M. Cai, and C. Xiong, "Research on human travel correlation for urban transport planning based on multisource data," *Sensors*, vol. 21, no. 1, p. 195, Jan. 2021.

[45] S. Guo et al., "Identifying the most influential roads based on traffic correlation networks," *EPJ Data Sci.*, vol. 8, no. 1, pp. 1–17, Dec. 2019.

[46] Z. Xiao et al., "TrajData: On vehicle trajectory collection with commodity plug-and-play OBU devices," *IEEE Internet Things J.*, vol. 7, no. 9, pp. 9066–9079, Sep. 2020.

[47] L. Liu, M. Zhao, M. Yu, M. A. Jan, D. Lan, and A. Taherkordi, "Mobility-aware multi-hop task offloading for autonomous driving in vehicular edge computing and networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2169–2182, Feb. 2023.

[48] L. Pons et al., "Effect of hyper-threading in latency-critical multithreaded cloud applications and utilization analysis of the major system resources," *Future Gener. Comput. Syst.*, vol. 131, pp. 194–208, Jun. 2022.


Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:38:28 UTC from IEEE Xplore. Restrictions apply.